

CSDP

Cerebras Distributed Streaming Protocol

Research Report and Formal Rust Protocol Specification

Version 1.0-draft

Research baseline: 2026-10-03

Document status: Proposed architecture / experimental protocol; not an official Cerebras specification.

Core thesis: keep mature RPC/schema mechanisms for control and metadata, but remove generic serialization from the bulk tensor data path. CSDP describes tensors once and streams raw, chunk-addressable bytes over an RDMA/low-latency transport.

Primary implementation language: Rust

Primary data-plane transport target: RoCE v2 / RDMA, with TCP compatibility and a Direct-Wafer transport adapter where platform APIs permit.

Primary serialization research: gRPC/Protobuf, MessagePack, BSON, CBOR, Cap'n Proto, FlatBuffers, Serde/Postcard, and rkyv.

Evidence note

Public Cerebras sources document MemoryX/SwarmX roles, Weight Streaming, CS-3/WSE-3 capabilities, and the CS-4/Nexus I/O direction. The public materials reviewed do not disclose enough detail to establish Cerebras' internal MemoryX or SwarmX wire format; therefore, CSDP is presented explicitly as a new protocol proposal.

Executive Summary

CSDP is a proposal for a high-throughput distributed computing and data-streaming protocol tailored to the published characteristics of Cerebras systems. The protocol separates management/control, stream metadata, and bulk tensor transport. The bulk path uses a fixed 64-byte base header followed by raw tensor bytes, allowing chunk-level addressing, CRC32C validation, credit-based flow control, and direct placement into DMA/RDMA-capable buffers.

Contents

1. Executive Summary
 - 1.1 Public architecture baseline
 - 1.2 The protocol opportunity
 - 1.3 Core design decision
 - 1.4 Recommended protocol stack
2. Scope, Evidence, and Assumptions
 - 2.1 Scope
 - 2.2 What is public and what is proposed
 - 2.3 Engineering assumptions
3. Cerebras System Architecture Relevant to CSDP
 - 3.1 WSE architectural implications
 - 3.2 Weight Streaming
 - 3.3 SwarmX as an active communication layer
 - 3.4 CS-4 and Nexus
 - 3.5 Disaggregated inference
4. Requirements for a Next-Generation Protocol
 - 4.1 Data-plane requirements
 - 4.2 Control-plane requirements
 - 4.3 Distributed-system requirements
5. Binary Encoding and Serialization Research
 - 5.1 Protocol Buffers
 - 5.2 gRPC
 - 5.3 MessagePack
 - 5.4 BSON
 - 5.5 CBOR
 - 5.6 Cap'n Proto
 - 5.7 FlatBuffers
 - 5.8 Serde
 - 5.9 Postcard
 - 5.10 rkyv
 - 5.11 Comparative matrix

6. Proposed CSDP Architecture

6.1 Three-plane model

Plane A: Management

Plane B: Stream control

Plane C: Bulk data

6.2 Why a fixed header

6.3 Descriptor-first streaming

7. MemoryX Data Model Proposal

7.1 Logical object model

7.2 Extent storage

7.3 Hot and cold tiers

7.4 Zero-copy target

8. Distributed Compute and Instance Clustering

8.1 Node identity

8.2 Epoch fencing

8.3 Logical job group

8.4 Topology abstraction

9. Collective Protocol

9.1 Operations

9.2 In-network execution

9.3 Reduction semantics

9.4 Hierarchical collectives

10. Flow Control and Streaming

10.1 Receiver credits

10.2 Acceptance states

10.3 Double and triple buffering

10.4 Chunk sizing

11. Reliability, Integrity, and Recovery

11.1 Frame integrity

11.2 Object integrity

11.3 Partial retransmission

11.4 Idempotence

11.5 Stale epoch handling

12. Security Model

12.1 Deployment profiles

Trusted-Fabric

Secure-Data-Plane

12.2 Authentication

- 12.3 Authorization
- 12.4 Replay protection
- 13. Performance Model and Benchmark Plan
 - 13.1 What should be benchmarked
 - 13.2 Measurements
 - 13.3 Primary KPI
 - 13.4 Hypothesis
- 14. Protocol Versioning and Compatibility
- 15. Formal CSDP/1.0 Protocol Specification
 - 15.1 Transport independence
 - 15.2 Byte order
 - 15.3 Base frame
 - 15.4 Exact 64-byte base header
 - 15.5 Magic value
 - 15.6 Flags
 - 15.7 MessageType
 - 15.8 Status codes
 - 15.9 Node roles
 - 15.10 Data types
 - 15.11 Stream classes
 - 15.12 Compression identifiers
 - 15.13 Collective operations
 - 15.14 HELLO payload
 - 15.15 STREAM_OPEN payload
 - 15.16 TENSOR_OPEN payload
 - 15.17 DATA semantics
 - 15.18 CREDIT_UPDATE payload
 - 15.19 DATA_ACK payload
 - 15.20 NACK payload
 - 15.21 CollectiveOpen payload
 - 15.22 Error handling
- 16. CSDP Session and Stream State Machines
 - 16.1 Session states
 - 16.2 Stream states
 - 16.3 Tensor states
- 17. Rust Reference Implementation Architecture
 - 17.1 Crate layout
 - 17.2 Core dependencies
 - 17.3 Zero-copy principles in Rust

17.4 Unsafe code policy	
18. Reference Rust Wire Module	
19. CSDP Test Vector	
20. Reference Processing Pipeline	
20.1 Sender	
20.2 Receiver	
21. MemoryX Streaming Scheduler Proposal	
22. Future Extensions	
22.1 Sparse tensor encoding	
22.2 Quantized formats	
22.3 GPU/accelerator memory registration	
22.4 Programmable fabric operations	
22.5 Direct-Wafer specialization	
23. Risks and Open Questions	
23.1 Public-information gap	
23.2 Serialization benchmark risk	
23.3 RDMA deployment complexity	
23.4 Security/performance trade-off	
23.5 Determinism versus collective performance	
24. Implementation Roadmap	
Phase 1 - Reference protocol	
Phase 2 - MemoryX simulator	
Phase 3 - RDMA prototype	
Phase 4 - Collective engine	
Phase 5 - Hardware integration	
25. Final Design Position	
Appendix A. Compact CSDP Wire Reference	
Appendix B. Recommended Default Parameters	
Appendix C. Glossary	
Appendix D. References	
Document Change Log	

Abstract

Cerebras Systems takes a different approach to large-scale AI acceleration from conventional GPU clusters: the Wafer-Scale Engine places a very large compute fabric and distributed SRAM on a single wafer, while system-level scale-out historically relies on disaggregated MemoryX storage and SwarmX broadcast/reduce communication. Public Cerebras material now describes a further evolution in CS-4/Nexus: three WSE-3 Turbo processors per rack-scale system, programmable wafer I/O, standards-based RoCE v2 RDMA over Ethernet, switch-free Direct Wafer Links, and wafer-to-wafer latency as low as 2 microseconds.

This report proposes **CSDP (Cerebras Distributed Streaming Protocol)** as a new transport/application protocol for that class of architecture. The central design decision is to avoid treating large tensors as ordinary serialized application objects. Instead, CSDP separates:

1. a control and metadata plane, where schema-based binary serialization is useful;
2. a stream-control plane, where compact Rust data structures describe tensors, storage extents, collectives, credits, epochs, and capabilities; and
3. a bulk tensor data plane, where each frame consists of a fixed binary header followed by raw tensor bytes that can be placed directly into registered DMA memory.

The proposal evaluates Protocol Buffers, MessagePack, BSON, CBOR, Cap'n Proto, FlatBuffers, Serde-based Postcard, Bincode-like Rust formats, and rkyv. The resulting architecture uses gRPC/Protobuf where ecosystem compatibility and management semantics matter, while allowing CSDP metadata to use a Rust-native Postcard/Serde profile and a portable CBOR profile. Cap'n Proto or FlatBuffers remain viable alternatives for cross-language zero-copy descriptors. rkyv is proposed primarily for local MemoryX metadata and persistent indexes. For the tensor hot path, serialization is intentionally removed rather than merely replaced by a different object format.

This document contains the complete proposed CSDP/1.0 wire model, exact 64-byte frame header, message taxonomy, control schemas, stream state machines, credit-based flow control, chunking, integrity, fault handling, cluster membership, topology-aware collectives, MemoryX storage model, security model, and a Rust reference implementation skeleton.

1. Executive Summary

1.1 Public architecture baseline

Cerebras public documentation describes a Wafer-Scale Cluster as a combination of CS systems, MemoryX, SwarmX, input-preprocessing infrastructure, and management/orchestration components. In the Weight Streaming execution mode, MemoryX stores model weights; SwarmX broadcasts weights to the compute systems and reduces gradients in the reverse direction. [R1] [R2][R3]

The WSE-3 inside CS-3 has 4 trillion transistors, 900,000 AI-optimized compute cores, and 44 GB of on-wafer SRAM. Cerebras states that CS-3 clusters can scale to 2,048 systems and publishes MemoryX configurations extending from tens of terabytes to 1,200 TB for hyperscale deployments. [R4][R5]

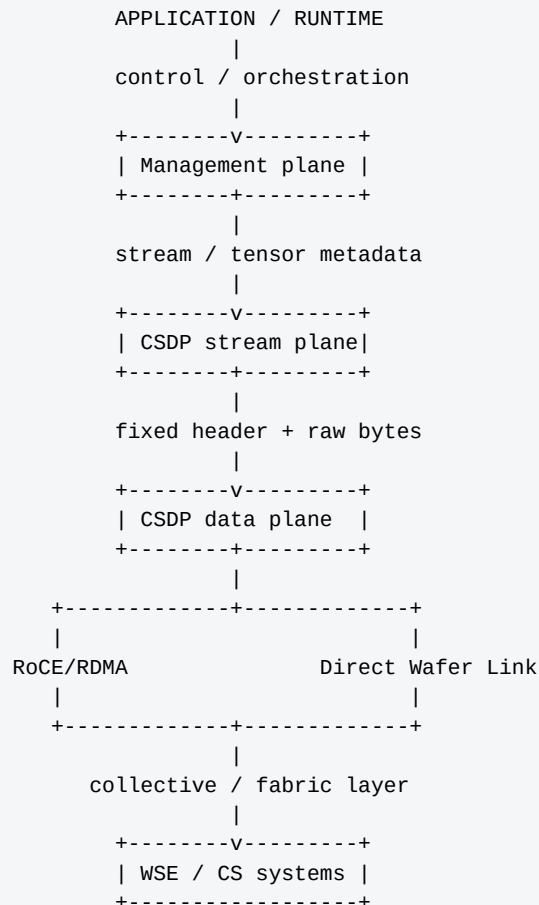
In August 2026 Cerebras introduced CS-4 and the Nexus rack-scale architecture. CS-4 combines three WSE-3 Turbo processors. Cerebras publishes 250 PFLOPS of AI compute and 43.2 PB/s of memory bandwidth per WSE-3T, 2.4 Tb/s of off-wafer bandwidth per wafer, 7.2 Tb/s per CS-4 rack-scale solution, and wafer-to-wafer latency as low as 2 microseconds. The new programmable

Wafer I/O Module supports RoCE v2 RDMA over Ethernet and Direct Wafer Links that can connect wafers without an intervening switch. [R6][R7]

Cerebras has also publicly described heterogeneous disaggregated inference, including an architecture in which a prefill engine produces KV cache and the Cerebras WSE performs decode. AWS and Cerebras described Trainium-to-Cerebras transfer using AWS EFA, while the AMD/Cerebras collaboration describes a future Helios + WSE split. [R8][R9]

1.2 The protocol opportunity

The architectural trend is toward a more general **data movement fabric** rather than a weight-only transport path:



The proposal therefore treats CSDP as a **protocol suite**, not merely a serializer replacement.

1.3 Core design decision

The major improvement over a generic gRPC + Protobuf bulk-data path is not expected to come from choosing MessagePack instead of Protobuf. A Protobuf `bytes` field already carries raw binary data as a length-delimited payload. [R10]

The high-value optimization is instead:

Describe a tensor once; stream its raw bytes as independently addressable chunks without reconstructing a serialized tensor object.

This enables direct placement into pre-registered, NUMA-local memory and makes RDMA/DMA and scatter-gather transfer first-class protocol operations.

1.4 Recommended protocol stack

Layer	Recommended mechanism	Purpose
Management/control	gRPC + Protobuf, or FlatBuffers/Cap'n Proto where justified	Jobs, resource control, APIs, scheduling, topology
CSDP session/metadata	Postcard + Serde for Rust; CBOR compatibility profile	Compact descriptors and control messages
Local MemoryX metadata	rkyv-style zero-copy archive	Persistent index, manifests, extent metadata
Bulk tensors	CSDP fixed frame + raw bytes	Zero-copy/low-copy high-bandwidth data movement
Fabric	RoCE v2/RDMA, Direct-Wafer adapter, TCP fallback	Physical/transport delivery
Integrity	CRC32C per frame + optional cryptographic digest per object	Fast corruption detection + end-to-end verification
Security	Trusted-fabric profile + optional AEAD profile	Data-plane protection

2. Scope, Evidence, and Assumptions

2.1 Scope

This report covers:

- distributed compute instances and cluster membership;
- wafer/system/rack-scale topology representation;
- MemoryX-oriented storage and streaming;
- weight, gradient, activation, KV-cache, checkpoint, and optimizer-state movement;
- broadcast, reduce, gather, scatter, and all-gather operations;
- flow control and backpressure;
- zero-copy / low-copy transfer;
- fault isolation and resumable chunk transfer;
- security and integrity;
- a formal CSDP/1.0 wire specification for a Rust implementation.

2.2 What is public and what is proposed

The following are treated as published facts where cited:

- WSE-3/CS-3 characteristics;
- MemoryX and SwarmX's role in Weight Streaming;
- CS-3 cluster scale and MemoryX capacity options;
- CS-4/Nexus architecture and published I/O capabilities;
- current public descriptions of disaggregated inference.

The following are **not assumed to be public Cerebras internals**:

- MemoryX's private storage engine implementation;
- the exact internal MemoryX wire encoding;

- the private SwarmX packet format;
- private Cerebras firmware protocol fields;
- internal routing or scheduling algorithms not documented publicly.

This report intentionally does not claim that Cerebras currently uses gRPC, Protocol Buffers, MessagePack, or any other format inside MemoryX or SwarmX. The protocol described here is a new proposal designed to fit the published architectural characteristics.

2.3 Engineering assumptions

The proposal assumes that a deployment can provide at least one of:

- reliable RDMA transport such as RoCE v2;
- a trusted low-latency proprietary or device-local fabric;
- TCP as a compatibility transport.

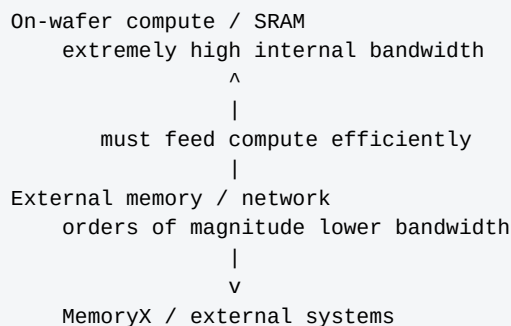
It also assumes that the compute and storage endpoints can expose buffers suitable for asynchronous I/O and, where supported, DMA/RDMA placement.

3. Cerebras System Architecture Relevant to CSDP

3.1 WSE architectural implications

The Cerebras architecture is fundamentally different from a conventional GPU server because the principal working memory is distributed SRAM on the wafer rather than a small number of external HBM stacks. Public WSE-3 material identifies 44 GB of SRAM and 900,000 cores per wafer. [R4]

This architecture creates a strong asymmetry:



Any external stream therefore has to be carefully overlapped with compute. The protocol cannot remove the physical bandwidth gap, but it can minimize additional software overhead and avoid turning external bandwidth into CPU serialization work.

3.2 Weight Streaming

Public Cerebras documentation describes the weight-streaming training sequence as follows: a layer's weights are brought from MemoryX to the WSE; with multiple systems, SwarmX broadcasts the weights; the WSEs compute in parallel; gradients are then sent back and reduced; MemoryX applies the update. [R1][R2]

The protocol consequence is that a layer is a natural streaming object:

```

TensorDescriptor(layer N)
|
  
```

```

+-- chunk 0
+-- chunk 1
+-- chunk 2
...
+-- chunk N

```

The descriptor is metadata. The chunks are bulk data.

3.3 SwarmX as an active communication layer

Cerebras describes SwarmX as more than a passive network: it implements broadcast on the weight path and reduction on the gradient path, using a tree topology for modular scale-out. [R3]

CSDP therefore makes collectives explicit protocol objects rather than implicit side effects of point-to-point messages.

3.4 CS-4 and Nexus

CS-4 materially expands the interconnect design space. Cerebras describes three WSE-3T processors in a rack-scale Nexus system, programmable I/O, RoCE v2, and Direct Wafer Links. The published bandwidth figures are 2.4 Tb/s off-wafer per wafer and 7.2 Tb/s for the three-wafer system, with direct wafer-to-wafer latency as low as 2 microseconds. [R6][R7]

This motivates a protocol that is:

- transport-independent at the logical API level;
- strongly optimized for RDMA;
- capable of mapping collective operations onto hierarchical topology;
- able to pass descriptors between heterogeneous endpoints;
- capable of handling large model and KV-cache transfers.

3.5 Disaggregated inference

AWS and Cerebras have described disaggregating prefill and decode, with Trainium producing KV cache and Cerebras handling decode. The AMD/Cerebras collaboration similarly targets heterogeneous disaggregation. [R8][R9]

This widens the useful CSDP object model from weights and gradients to:

```

WEIGHT
GRADIENT
ACTIVATION
KV_CACHE
CHECKPOINT
OPTIMIZER_STATE
CONTROL

```

4. Requirements for a Next-Generation Protocol

The protocol should satisfy the following requirements.

4.1 Data-plane requirements

1. No mandatory object-level serialization of tensors.
2. Fixed-size base header for predictable parsing.

3. Direct access to raw tensor bytes.
4. Scatter-gather capable.
5. Chunk-level addressing and replay.
6. DMA/RDMA-friendly buffer placement.
7. Minimal allocations in the hot path.
8. Explicit data type, shape, layout, and generation metadata.
9. Large object support beyond the limits of ordinary RPC messages.
10. Fast integrity checks.

4.2 Control-plane requirements

1. Schema-evolvable descriptors.
2. Explicit version and capability negotiation.
3. Cluster membership and fencing.
4. Topology distribution.
5. Stream lifecycle management.
6. Credit-based flow control.
7. Collective operation definitions.
8. Error propagation and cancellation.
9. Health/heartbeat primitives.
10. Cross-language interoperability options.

4.3 Distributed-system requirements

- no stale-node participation after an epoch change;
- idempotent transfer operations;
- partial retransmission;
- deterministic collective mode as an option;
- topology-aware placement;
- graceful node replacement;
- restartable checkpoints;
- bounded buffering.

5. Binary Encoding and Serialization Research

5.1 Protocol Buffers

Protocol Buffers encodes fields as field-number/wire-type pairs. bytes, strings, embedded messages, and packed repeated fields use the length-delimited wire type. Public Protobuf documentation states that serialized messages are limited to below 2 GiB in common implementations. [R10]

For tensors represented as a bytes field, the bulk is already close to raw data. Therefore:

```
protobuf metadata + bytes(tensor)
```

is not fundamentally inefficient in wire density.

The potential overhead comes from the surrounding RPC and application stack: message construction, framework buffering, object lifetime, streaming semantics, and additional copies.

5.2 gRPC

gRPC provides mature streaming, connection management, flow control, cancellation, load balancing, and cross-language tooling. The gRPC performance guidance recommends reusing channels and using streaming for long-lived flows, while also noting that long-lived streams can create load-balancing and scalability challenges. gRPC flow control is designed to protect fast producers from overwhelming receivers. [R11][R12]

Therefore gRPC remains appropriate for control operations, but the CSDP design intentionally avoids making HTTP/2 RPC streams the only bulk-data abstraction.

5.3 MessagePack

MessagePack defines compact binary representations for integers, strings, maps, arrays, binary values, and extensions. Its binary format is well suited to generic metadata and portable tools. [R13]

However, for a large tensor, MessagePack still presents the tensor as a binary value inside a generic object representation. That is better than text but does not remove the conceptual serialization boundary.

Recommended use: control metadata, tooling, manifests.

5.4 BSON

BSON is a binary document format with a dedicated binary value mechanism. The BSON specification also includes vector-related binary subtypes. [R14]

BSON is useful where document-oriented interoperability or MongoDB tooling matters, but its document model is not a natural match for a high-rate fixed-header tensor transport. It introduces more semantic structure than required by the hot data path.

Recommended use: external metadata or archival/document interoperability, not the core tensor stream.

5.5 CBOR

CBOR is an IETF standard binary representation with compact scalar types, arrays, maps, tags, and byte strings. Its explicit byte-string type and extensibility make it strong for portable control messages. [R15]

Recommended use: portable CSDP control profile, configuration/manifests, ecosystem integration.

5.6 Cap'n Proto

Cap'n Proto is designed around a serialized representation that can be accessed without a traditional unpacking step. Its encoding includes a packed variant that compresses zero bytes efficiently. [R16]

This is a strong candidate when a cross-language protocol needs low-copy structured descriptors.

Recommended use: alternative CSDP descriptor profile in mixed Rust/C++/Go environments.

5.7 FlatBuffers

FlatBuffers provides access to serialized data without unpacking and supports multiple programming languages, including Rust. Its Rust documentation describes read-only access directly from the original buffer and Send + Sync generated views. [R17]

Recommended use: portable zero-copy descriptor schema if the protocol must be shared widely across non-Rust implementations.

5.8 Serde

Serde is a Rust framework connecting Rust data structures to serialization formats. Its trait-based design can be compile-time optimized in many cases, avoiding the reflection overhead common to runtime-driven serialization. [R18]

Serde is therefore an appropriate programming model for CSDP control structures, but Serde itself is not a wire encoding.

5.9 Postcard

Postcard is a compact Serde serializer intended for resource-efficient Rust systems. Its documentation states that its format has been stable since Postcard 1.0 and uses compact variable-length encoding. [R19]

The Rust CSDP profile uses Postcard for control objects because it gives a simple, compact, Rust-native wire encoding without forcing tensors through a generic map/document representation.

5.10 rkyv

rkyv is a Rust zero-copy deserialization/archive framework. Its archive format replaces types with archived representations and provides stable layouts and byte orders; it is specifically designed for access without reconstructing a separate ordinary object representation. [R20][R21]

This makes rkyv compelling for MemoryX metadata, local indexes, and memory-mapped persistent state.

5.11 Comparative matrix

Format	Wire efficiency	Decode cost	Zero-copy access	Cross-language	Schema evolution	CSDP role
gRPC + Protobuf	High	Medium	Limited for ordinary objects	Excellent	Excellent	Control plane / compatibility
MessagePack	High	Medium	Limited	Excellent	Good with discipline	Portable metadata
BSON	Medium	Medium	Limited	Excellent	Good	Documents / tooling
CBOR	High	Medium	Limited	Excellent	Excellent	Portable metadata
Cap'n Proto	High	Low	Strong	Excellent	Strong	Descriptor plane
FlatBuffers	High	Low	Strong	Excellent	Strong	Descriptor plane
Serde +	Very high for	Low	Borrowing	Moderate	Schema	Rust control

Format	Wire efficiency	Decode cost	Zero-copy access	Cross-language	Schema evolution	CSDP role
Postcard	small Rust messages		possible; not generic archived memory		discipline required	profile
rkyv	Very high for local archive	Very low	Strong	Low to moderate	Requires archive discipline	MemoryX local metadata
Custom CSDP frame	Near raw	Minimal	Strong	Protocol-dependent	Explicit versioning	Tensor data plane

The matrix is qualitative. No numeric throughput is claimed here without a hardware-specific benchmark.

6. Proposed CSDP Architecture

6.1 Three-plane model

CSDP separates three functions.

Plane A: Management

May continue to use existing gRPC/Protobuf or another mature RPC framework.

Plane B: Stream control

Opens sessions, describes tensors, configures credits, negotiates transport features, and manages collectives.

Plane C: Bulk data

Carries raw bytes with a fixed binary framing structure.

6.2 Why a fixed header

The base header is exactly 64 bytes. This is large enough for transport/session/object identifiers while remaining predictable and cache-friendly.

Header overhead is negligible for normal tensor chunk sizes:

Payload	64-byte header overhead
1 KiB	6.25%
64 KiB	0.0977%
1 MiB	0.00610%
16 MiB	0.000381%
64 MiB	0.0000954%
256 MiB	0.0000238%
1 GiB	0.00000596%

The table is arithmetic, not a benchmark. It illustrates why the important design constraint is **avoiding per-object/per-tensor processing**, not eliminating every few bytes of framing.

6.3 Descriptor-first streaming

A tensor is opened once:

```

TENSOR_OPEN
  object_id
  tensor_id
  generation
  dtype
  shape
  strides/layout
  total_bytes
  chunk_size
  checksum algorithm

DATA
  chunk 0
DATA
  chunk 1
DATA
  chunk 2
...
DATA
  chunk N

TENSOR_FIN
  final digest

```

This avoids repeating dtype, shape, names, and other metadata on every frame.

7. MemoryX Data Model Proposal

7.1 Logical object model

MemoryX is modeled as a tiered tensor object store:

```

Model
  +-- Generation
    +-- Tensor
      +-- Extent 0
      +-- Extent 1
      +-- Extent 2

Checkpoint
  +-- Generation
    +-- Tensor references / immutable extents

```

Objects are immutable once committed. A new training step creates a new generation or replaces extents according to deployment policy.

7.2 Extent storage

A tensor should be represented by large sequential extents instead of many tiny records:

```

pub struct TensorExtent {
  pub tensor_id: u64,
  pub generation: u64,

```

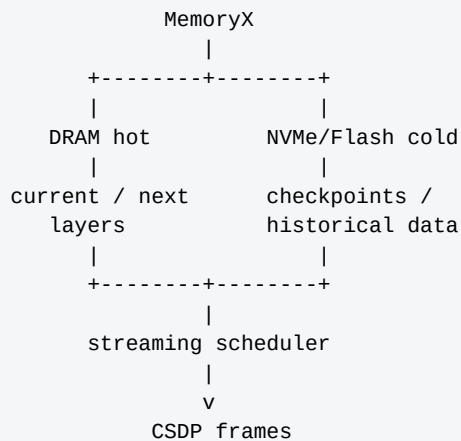
```

pub storage_tier: StorageTier,
pub segment_id: u64,
pub offset: u64,
pub length: u64,
pub digest: [u8; 32],
}

```

The index can be persisted as an rkyv archive or a compact Serde/Postcard structure.

7.3 Hot and cold tiers



The scheduler should prefetch based on the execution plan and receiver credit.

7.4 Zero-copy target

The preferred path is:

```

NVMe / storage
  |
  v
registered NUMA-local buffer
  |
  v
RDMA DMA
  |
  v
CSDP / fabric
  |
  v
WSE input / staging memory

```

rather than a repeated chain of object materialization and reserialization.

8. Distributed Compute and Instance Clustering

8.1 Node identity

Each participating endpoint has:

- a stable node_id;
- an incarnation counter that changes after restart;
- a cluster epoch supplied by the control plane;

- a node_role;
- transport and capability bits.

8.2 Epoch fencing

A receiver MUST reject stream operations from a stale cluster epoch.

This prevents a partitioned or restarted instance from injecting old gradients, checkpoints, or tensor data into a newer training job.

8.3 Logical job group

A distributed execution group contains:

```
job_id
cluster_epoch
world_size
rank
collective_group_id
node list
stream mappings
```

The data path does not depend on a centralized scheduler after the stream has been established, except where a deployment explicitly chooses centralized control.

8.4 Topology abstraction

The control plane distributes a logical topology:

```
Cluster
|
+-- FabricGroup A
|   +-- node 0
|   +-- node 1
|   +-- node 2
|
+-- FabricGroup B
    +-- node 3
    +-- node 4
    +-- node 5
```

A topology-aware planner can then select a tree, ring, hierarchical tree, or direct point-to-point path.

9. Collective Protocol

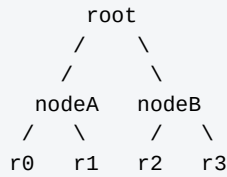
9.1 Operations

CSDP defines the following logical collectives:

```
pub enum CollectiveOp {
    Broadcast,
    Reduce,
    AllReduce,
    Gather,
    Scatter,
    AllGather,
}
```

9.2 In-network execution

For a broadcast, one source can inject a tensor once and fabric nodes replicate it:



For reduction, the direction is reversed and intermediate nodes may combine partial results.

9.3 Reduction semantics

A reduction operation **MUST** declare:

- data type;
- operator;
- whether the operator is associative;
- whether the implementation may reorder operations;
- whether deterministic ordering is required.

Supported baseline operators:

```
SUM
MAX
MIN
```

For floating-point SUM, deterministic mode uses a fixed topology and reduction order. Non-deterministic mode may allow topology-dependent ordering for throughput.

9.4 Hierarchical collectives

The recommended implementation is hierarchical:

```
wafer / local fabric
      |
system / backpack
      |
rack group
      |
cluster group
```

This is not a claim about current Cerebras internal topology; it is the protocol abstraction needed to map a collective onto whatever topology is available.

10. Flow Control and Streaming

10.1 Receiver credits

CSDP uses explicit byte and chunk credits.

A sender **MUST NOT** send data that exceeds the current stream credit window.

The receiver replenishes credits only after the corresponding data has reached the receiver's configured acceptance state.

10.2 Acceptance states

A stream can negotiate one of:

```

BUFFERED      = data copied into a receive-owned buffer
DMA_ACCEPTED  = data has reached its DMA destination
CONSUMED      = downstream processing has accepted the data

```

DMA_ACCEPTED is recommended for large tensor streams where the consumer can overlap computation with subsequent transfers.

10.3 Double and triple buffering

A Weight Streaming implementation can use:

```

buffer A -> current layer
buffer B -> next layer
buffer C -> optional look-ahead

```

The aim is to hide transfer time behind compute rather than allowing the protocol to become a synchronization barrier.

10.4 Chunk sizing

CSDP does not mandate one universal payload size because the ideal chunk size depends on transport, NIC, memory registration, NUMA topology, and implementation.

The protocol SHOULD support at least 64 KiB to 64 MiB payloads. An implementation SHOULD benchmark 1 MiB, 4 MiB, 8 MiB, 16 MiB, and 64 MiB for its target system.

The benchmark, not a protocol constant, should determine the default.

11. Reliability, Integrity, and Recovery

11.1 Frame integrity

Each data frame carries CRC32C over the payload. CRC32C is intended as a fast corruption detector, not as a security primitive.

11.2 Object integrity

A tensor or checkpoint object SHOULD also have an end-to-end cryptographic digest, such as BLAKE3 or SHA-256.

The frame checksum detects local corruption; the object digest verifies the reconstructed object.

11.3 Partial retransmission

Chunks are individually identified by (object_id, chunk_id).

A receiver can maintain a bitmap:

```

1111111101111111
      ^
    missing chunk

```

Only missing chunks need retransmission.

11.4 Idempotence

A duplicate DATA frame MUST be safe to discard after successful validation of the existing chunk. A retried control operation MUST use the same operation identifier so that a receiver can recognize repeated requests.

11.5 Stale epoch handling

A receiver MUST return STALE_EPOCH if the stream belongs to an obsolete cluster epoch.

12. Security Model

12.1 Deployment profiles

CSDP defines two security profiles.

Trusted-Fabric

Used where the underlying fabric is physically controlled and access is isolated. Integrity is still enforced with CRC32C and optional object digests.

Secure-Data-Plane

Uses session authentication and optional AEAD for payload protection. The cipher suite is negotiated during session establishment.

The protocol does not assume that a proprietary wafer link is inherently secure.

12.2 Authentication

The HELLO/WELCOME exchange identifies the node and the protocol capabilities. Authentication can be performed using certificates, pre-shared keys, or an external identity mechanism supplied by the deployment.

12.3 Authorization

A session carries an authorization context. A receiver MUST verify that the peer is allowed to read or write each stream object.

12.4 Replay protection

session_id, epoch, and monotonically increasing sequence values form the minimum replay-detection state. Secure profile implementations SHOULD bind AEAD nonces to session and sequence information.

13. Performance Model and Benchmark Plan

13.1 What should be benchmarked

A meaningful benchmark compares end-to-end tensor movement rather than serializing a toy struct.

Workloads:

```
1 KiB
64 KiB
1 MiB
```

```

16 MiB
64 MiB
256 MiB
1 GiB

```

Data types:

```

FP32
FP16
BF16
FP8
INT8

```

Formats:

```

gRPC + Protobuf bytes
MessagePack
CBOR
Cap'n Proto
FlatBuffers
Serde + Postcard
rkyv
CSDP raw frame

```

13.2 Measurements

Measure:

- serialization bandwidth;
- deserialization bandwidth;
- CPU cycles per byte;
- allocations;
- memcpy volume;
- NUMA traffic;
- wire overhead;
- end-to-end latency;
- p50/p99 completion latency;
- RDMA throughput;
- receiver backpressure;
- percentage of compute time stalled waiting for data.

13.3 Primary KPI

The primary KPI should be:

```

Useful tensor bytes delivered and accepted
-----
CPU cost + memory-copy cost + network overhead + compute stalls

```

The fastest generic serializer is not necessarily the fastest system protocol.

13.4 Hypothesis

For large tensors, CSDP should show its biggest advantage by reducing copies and object handling, while the relative difference between binary serialization formats becomes small because the tensor payload itself dominates the byte count.

This hypothesis must be validated by benchmark; it is not presented as an experimental result.

14. Protocol Versioning and Compatibility

CSDP uses `version_major` and `version_minor` in the base frame header.

Rules:

- Major versions are wire-incompatible unless explicitly bridged.
- Minor versions **MUST** preserve the meaning of existing fields.
- Unknown extension fields **MAY** be skipped when the `HAS_EXT` flag is set.
- Implementations **MUST** advertise supported message types and features.
- A receiver **MUST** reject unsupported required features rather than silently downgrading.

The stream-control payload includes a schema identifier:

```
schema_id = "cerebras.csdp.control.v1"
```

The Rust profile uses Postcard. A future cross-language profile can use CBOR without changing the bulk data framing.

15. Formal CSDP/1.0 Protocol Specification

*The remainder of this section is normative for the proposed CSDP/1.0 draft. The key words ****MUST****, ****MUST NOT****, ****REQUIRED****, ****SHOULD****, ****SHOULD NOT****, and ****MAY**** are to be interpreted in their ordinary RFC 2119-style sense.*

15.1 Transport independence

CSDP defines messages independent of the underlying transport.

A transport adapter **MUST** provide:

```
send(bytes)
receive(bytes)
optional ordered reliable delivery
optional zero-copy receive buffer
optional RDMA work-request semantics
```

The initial profiles are:

```
CSDP/RDMA    -> RoCE v2 or compatible RDMA transport
CSDP/TCP     -> length-aware byte stream
CSDP/DWL     -> Direct-Wafer Link adapter
```

15.2 Byte order

All fixed-width integer fields in the CSDP base header are encoded **little-endian**.

This is a deliberate design choice optimized for the expected host implementation environment. Implementations using a different native endianness **MUST** convert at the protocol boundary. Postcard control payloads use the Postcard wire representation defined by the selected Postcard profile.

15.3 Base frame

Every frame consists of:

```
+-----+ 64 bytes
| CSDP base header |
+-----+
| optional extensions | header_len - 64
+-----+
| payload             | payload_len
+-----+
```

header_len **MUST** be a multiple of 8 bytes and **MUST** be at least 64.

15.4 Exact 64-byte base header

Offset	Size	Field	Meaning
0	4	magic	ASCII CSDP = 43 53 44 50
4	1	version_major	Protocol major version
5	1	version_minor	Protocol minor version
6	1	header_len_words	Header length / 8
7	1	flags	Frame flags
8	2	message_type	MessageType enum
10	2	status	StatusCode; normally zero
12	8	session_id	Session identifier
20	4	stream_id	Stream identifier
24	8	sequence	Per-stream sequence number
32	8	object_id	Tensor/checkpoint/control object identifier
40	4	chunk_id	Zero-based object chunk identifier
44	4	reserved	MUST be zero in v1
48	8	payload_len	Payload length in bytes
56	4	payload_crc32c	CRC32C of payload
60	4	header_crc32c	CRC32C of bytes 0..59

The CRC32C is computed over the exact transmitted header with the `header_crc32c` field treated as zero.

15.5 Magic value

The first four bytes **MUST** equal ASCII:

```
43 53 44 50
```

representing CSDP.

15.6 Flags

```
pub mod flags {
  pub const FIN: u8      = 0x01;
  pub const ACK_REQ: u8  = 0x02;
  pub const RETRANSMIT: u8 = 0x04;
  pub const HAS_EXT: u8   = 0x08;
  pub const ORDERED: u8   = 0x10;
  pub const UNORDERED: u8 = 0x20;
  pub const INLINE: u8    = 0x40;
  pub const RESERVED: u8  = 0x80;
}
```

ORDERED and UNORDERED **MUST NOT** both be set.

15.7 MessageType

```
#[repr(u16)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum MessageType {
  Hello          = 0x0001,
  Welcome        = 0x0002,
  Auth           = 0x0003,
  Capabilities    = 0x0004,

  StreamOpen     = 0x0010,
  StreamAccept   = 0x0011,
  StreamClose    = 0x0012,
  StreamReset    = 0x0013,

  TensorOpen     = 0x0020,
  TensorReady    = 0x0021,
  Data           = 0x0030,
  DataAck        = 0x0031,
  CreditUpdate   = 0x0032,
  TensorFin      = 0x0033,
  Nack           = 0x0034,

  CollectiveOpen  = 0x0040,
  CollectiveReady = 0x0041,
  CollectiveData  = 0x0042,
  CollectiveFin   = 0x0043,

  Heartbeat      = 0x0050,
  Pong           = 0x0051,

  Error          = 0x00ff,
}
```

Unknown message types MUST be rejected unless the implementation has a negotiated extension mechanism for them.

15.8 Status codes

```
#[repr(u16)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum StatusCode {
    Ok = 0,
    InvalidMessage = 1,
    Unsupported = 2,
    AuthFailed = 3,
    NotAuthorized = 4,
    FlowControl = 5,
    ChecksumFailed = 6,
    StaleEpoch = 7,
    NoResource = 8,
    Timeout = 9,
    Cancelled = 10,
    Duplicate = 11,
    OutOfOrder = 12,
    ProtocolError = 13,
    Internal = 14,
}
```

15.9 Node roles

```
#[repr(u8)]
pub enum NodeRole {
    Controller = 1,
    MemoryX = 2,
    Compute = 3,
    Fabric = 4,
    Gateway = 5,
    Hybrid = 6,
}
```

A node MAY have multiple logical roles if the deployment chooses a hybrid process.

15.10 Data types

```
#[repr(u8)]
pub enum DType {
    U8 = 1,
    I8 = 2,
    U16 = 3,
    I16 = 4,
    U32 = 5,
    I32 = 6,
    U64 = 7,
    I64 = 8,
    F8E4M3 = 20,
    F8E5M2 = 21,
    F16 = 22,
    BF16 = 23,
    F32 = 24,
    F64 = 25,
}
```

This list is extensible through capability negotiation.

15.11 Stream classes

```
#[repr(u8)]
pub enum StreamClass {
    Control      = 1,
    Weight       = 2,
    Gradient     = 3,
    Activation    = 4,
    KvCache      = 5,
    Checkpoint   = 6,
    OptimizerState = 7,
    Collective    = 8,
}
```

15.12 Compression identifiers

```
#[repr(u8)]
pub enum Compression {
    None      = 0,
    Zstd      = 1,
    Lz4       = 2,
    Quantized = 3,
    Sparse    = 4,
}
```

`Compression::Quantized` and `Compression::Sparse` are semantic representations rather than general-purpose byte compressors. Their parameters belong in the tensor descriptor.

15.13 Collective operations

```
#[repr(u8)]
pub enum CollectiveOp {
    Broadcast = 1,
    Reduce     = 2,
    AllReduce = 3,
    Gather    = 4,
    Scatter   = 5,
    AllGather = 6,
}
```

15.14 HELLO payload

The Rust profile encodes this structure using Serde + Postcard:

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct Hello {
    pub schema_version: u16,
    pub node_id: [u8; 16],
    pub incarnation: u64,
    pub cluster_epoch: u64,
    pub role: NodeRole,
    pub feature_bits: u64,
    pub max_frame_payload: u64,
    pub max_streams: u32,
    pub supported_dtypes: Vec<DType>,
    pub supported_compression: Vec<Compression>,
}
```

node_id is treated as opaque bytes. It MAY contain a UUID or another deployment-specific identifier.

15.15 STREAM_OPEN payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct StreamOpen {
    pub stream_class: StreamClass,
    pub cluster_epoch: u64,
    pub initial_credit_bytes: u64,
    pub initial_credit_chunks: u32,
    pub chunk_size: u32,
    pub ordered: bool,
    pub acceptance_mode: AcceptanceMode,
}

#[derive(Serialize, Deserialize, Debug, Clone, Copy)]
pub enum AcceptanceMode {
    Buffered,
    DmaAccepted,
    Consumed,
}
```

15.16 TENSOR_OPEN payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct TensorDescriptor {
    pub tensor_id: u64,
    pub generation: u64,
    pub dtype: DType,
    pub shape: Vec<u64>,
    pub strides: Vec<u64>,
    pub total_bytes: u64,
    pub chunk_size: u32,
    pub chunk_count: u32,
    pub compression: Compression,
    pub object_digest: Option<[u8; 32]>,
}
```

The descriptor MUST be accepted before DATA for the associated object_id unless an implementation has explicitly negotiated an inline descriptor profile.

15.17 DATA semantics

A DATA frame's payload is raw data or compressed chunk bytes according to the tensor descriptor.

The receiver MUST verify:

1. frame magic;
2. supported version;
3. header length;
4. frame status/flags consistency;
5. payload length;
6. payload CRC32C;
7. stream credit;

8. object/chunk identity;

9. cluster epoch validity.

The receiver MAY place the payload directly into the final destination buffer if the selected transport and layout allow it.

15.18 CREDIT_UPDATE payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct CreditUpdate {
    pub credit_bytes: u64,
    pub credit_chunks: u32,
}
```

Credits are additive. A receiver MUST NOT send a negative credit update.

15.19 DATA_ACK payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct DataAck {
    pub highest_contiguous_chunk: u32,
    pub selective_bitmap: Vec<u8>,
}
```

The bitmap is optional. It starts immediately after `highest_contiguous_chunk` and indicates subsequently received chunks.

15.20 NACK payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct Nack {
    pub missing_chunks: Vec<u32>,
    pub status: StatusCode,
}
```

A NACK SHOULD be used only where the underlying transport does not already provide reliable delivery or where application-level object recovery requires replay.

15.21 CollectiveOpen payload

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct CollectiveDescriptor {
    pub collective_id: u64,
    pub group_id: u64,
    pub operation: CollectiveOp,
    pub root_rank: Option<u32>,
    pub world_size: u32,
    pub rank: u32,
    pub tensor_id: u64,
    pub deterministic: bool,
}
```

15.22 Error handling

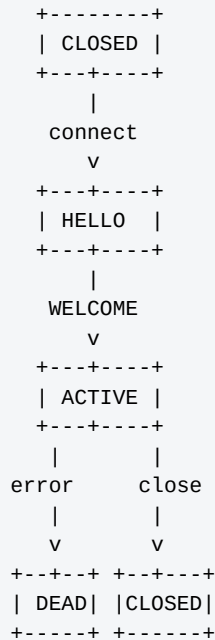
An ERROR frame MUST contain a non-zero status field and SHOULD include a human-readable diagnostic in its control payload.

A protocol error affecting only one stream **MUST** reset that stream without necessarily terminating the session.

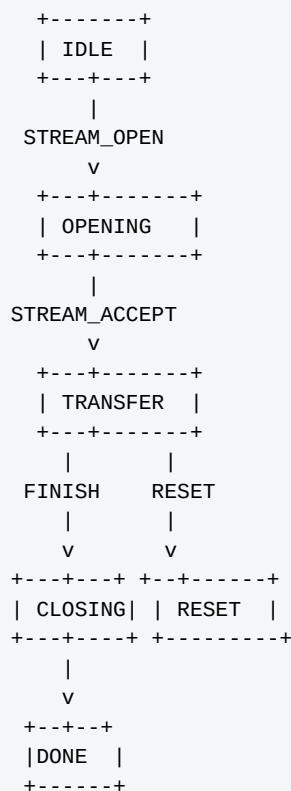
An authentication or protocol-version failure **MAY** terminate the session.

16. CSDP Session and Stream State Machines

16.1 Session states



16.2 Stream states



16.3 Tensor states

```
IDLE -> OPEN -> READY -> TRANSFERRING -> FIN -> COMMITTED
      \-> ERROR
```

17. Rust Reference Implementation Architecture

17.1 Crate layout

```
csdp/
  Cargo.toml
  src/
    lib.rs
    wire.rs
    frame.rs
    control.rs
    stream.rs
    tensor.rs
    collective.rs
    transport/
      mod.rs
      tcp.rs
      rdma.rs
      direct_wafer.rs
    storage/
      mod.rs
      index.rs
      extent.rs
      memoryx.rs
    checksum.rs
    error.rs
```

17.2 Core dependencies

A reference implementation can use:

```
[dependencies]
serde = { version = "1", features = ["derive"] }
postcard = "1"
tokio = { version = "1", features = ["rt-multi-thread", "net", "sync", "io-util"] }
bytes = "1"
crc32c = "0.6"
thiserror = "2"

# Optional:
# rkyv = "0.x"
# cbor4ii / minicbor = "..."
# flatbuffers = "..."
# blake3 = "..."
```

Version pins should be selected and audited by the implementation team; this report does not freeze third-party dependency versions.

17.3 Zero-copy principles in Rust

The code should favor:

```

&[u8]
&mut [u8]
Bytes
IoSlice
IoSliceMut

```

over repeated `Vec<u8>` allocation in the hot path.

Tensor descriptors are owned objects. Tensor payloads are borrowed buffers or DMA regions.

17.4 Unsafe code policy

The core protocol parser SHOULD remain safe Rust. Any unsafe block used to interface with RDMA verbs, device memory, or a vendor-specific SDK SHOULD be isolated behind a small transport abstraction.

18. Reference Rust Wire Module

The following is a minimal safe-Rust implementation of the base header encoding/decoding model. It is intentionally transport-neutral. A complete implementation must add stream management, Postcard control messages, RDMA adapters, storage integration, and security profiles.

```

use core::convert::TryFrom;

pub const HEADER_LEN: usize = 64;
pub const MAGIC: [u8; 4] = *b"CSDP";

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u16)]
pub enum MessageType {
    Hello = 0x0001,
    Welcome = 0x0002,
    Auth = 0x0003,
    Capabilities = 0x0004,
    StreamOpen = 0x0010,
    StreamAccept = 0x0011,
    StreamClose = 0x0012,
    StreamReset = 0x0013,
    TensorOpen = 0x0020,
    TensorReady = 0x0021,
    Data = 0x0030,
    DataAck = 0x0031,
    CreditUpdate = 0x0032,
    TensorFin = 0x0033,
    Nack = 0x0034,
    CollectiveOpen = 0x0040,
    CollectiveReady = 0x0041,
    CollectiveData = 0x0042,
    CollectiveFin = 0x0043,
    Heartbeat = 0x0050,
    Pong = 0x0051,
    Error = 0x00ff,
}

impl TryFrom<u16> for MessageType {
    type Error = &'static str;

    fn try_from(value: u16) -> Result<Self, Self::Error> {
        Ok(match value {
            0x0001 => Self::Hello,
            0x0002 => Self::Welcome,

```

```

        0x0003 => Self::Auth,
        0x0004 => Self::Capabilities,
        0x0010 => Self::StreamOpen,
        0x0011 => Self::StreamAccept,
        0x0012 => Self::StreamClose,
        0x0013 => Self::StreamReset,
        0x0020 => Self::TensorOpen,
        0x0021 => Self::TensorReady,
        0x0030 => Self::Data,
        0x0031 => Self::DataAck,
        0x0032 => Self::CreditUpdate,
        0x0033 => Self::TensorFin,
        0x0034 => Self::Nack,
        0x0040 => Self::CollectiveOpen,
        0x0041 => Self::CollectiveReady,
        0x0042 => Self::CollectiveData,
        0x0043 => Self::CollectiveFin,
        0x0050 => Self::Heartbeat,
        0x0051 => Self::Pong,
        0x00ff => Self::Error,
        _ => return Err("unknown message type"),
    })
}
}

#[derive(Debug, Clone, Copy)]
pub struct FrameHeader {
    pub version_major: u8,
    pub version_minor: u8,
    pub header_len_words: u8,
    pub flags: u8,
    pub message_type: MessageType,
    pub status: u16,
    pub session_id: u64,
    pub stream_id: u32,
    pub sequence: u64,
    pub object_id: u64,
    pub chunk_id: u32,
    pub payload_len: u64,
    pub payload_crc32c: u32,
}

impl FrameHeader {
    pub fn new(
        message_type: MessageType,
        session_id: u64,
        stream_id: u32,
        sequence: u64,
        object_id: u64,
        chunk_id: u32,
        payload: &[u8],
    ) -> Self {
        Self {
            version_major: 1,
            version_minor: 0,
            header_len_words: 8,
            flags: 0,
            message_type,
            status: 0,
            session_id,
            stream_id,
            sequence,
            object_id,
            chunk_id,
            payload_len: payload.len() as u64,

```

```

        payload_crc32c: crc32c(payload),
    }
}

pub fn encode(&self) -> [u8; HEADER_LEN] {
    let mut out = [0u8; HEADER_LEN];
    out[0..4].copy_from_slice(&MAGIC);
    out[4] = self.version_major;
    out[5] = self.version_minor;
    out[6] = self.header_len_words;
    out[7] = self.flags;
    out[8..10].copy_from_slice(&(self.message_type as u16).to_le_bytes());
    out[10..12].copy_from_slice(&self.status.to_le_bytes());
    out[12..20].copy_from_slice(&self.session_id.to_le_bytes());
    out[20..24].copy_from_slice(&self.stream_id.to_le_bytes());
    out[24..32].copy_from_slice(&self.sequence.to_le_bytes());
    out[32..40].copy_from_slice(&self.object_id.to_le_bytes());
    out[40..44].copy_from_slice(&self.chunk_id.to_le_bytes());
    out[44..48].fill(0);
    out[48..56].copy_from_slice(&self.payload_len.to_le_bytes());
    out[56..60].copy_from_slice(&self.payload_crc32c.to_le_bytes());
    out[60..64].fill(0);
    let checksum = crc32c(&out[..60]);
    out[60..64].copy_from_slice(&checksum.to_le_bytes());
    out
}

pub fn decode(bytes: &[u8]) -> Result<Self, &'static str> {
    if bytes.len() < HEADER_LEN {
        return Err("short header");
    }
    if bytes[0..4] != MAGIC {
        return Err("bad magic");
    }
    if bytes[6] < 8 {
        return Err("invalid header length");
    }

    let mut tmp = [0u8; HEADER_LEN];
    tmp.copy_from_slice(&bytes[..HEADER_LEN]);
    let expected = u32::from_le_bytes(tmp[60..64].try_into().unwrap());
    tmp[60..64].fill(0);
    if crc32c(&tmp[..60]) != expected {
        return Err("header checksum failure");
    }

    let message_type = MessageType::try_from(u16::from_le_bytes(
        bytes[8..10].try_into().unwrap(),
    ))
    .map_err(|_| "unknown message type")?;

    Ok(Self {
        version_major: bytes[4],
        version_minor: bytes[5],
        header_len_words: bytes[6],
        flags: bytes[7],
        message_type,
        status: u16::from_le_bytes(bytes[10..12].try_into().unwrap()),
        session_id: u64::from_le_bytes(bytes[12..20].try_into().unwrap()),
        stream_id: u32::from_le_bytes(bytes[20..24].try_into().unwrap()),
        sequence: u64::from_le_bytes(bytes[24..32].try_into().unwrap()),
        object_id: u64::from_le_bytes(bytes[32..40].try_into().unwrap()),
        chunk_id: u32::from_le_bytes(bytes[40..44].try_into().unwrap()),
        payload_len: u64::from_le_bytes(bytes[48..56].try_into().unwrap()),
        payload_crc32c: u32::from_le_bytes(bytes[56..60].try_into().unwrap()),
    })
}

```

```

    })
  }

  pub fn verify_payload(&self, payload: &[u8]) -> Result<(), &'static str> {
    if payload.len() as u64 != self.payload_len {
      return Err("payload length mismatch");
    }
    if crc32c(payload) != self.payload_crc32c {
      return Err("payload checksum failure");
    }
    Ok(())
  }
}

/// Portable, dependency-free CRC32C implementation for the reference module.
/// Production implementations SHOULD use a hardware-accelerated implementation.
pub fn crc32c(data: &[u8]) -> u32 {
  let mut crc = 0xffff_ffffu32;
  for &byte in data {
    crc ^= byte as u32;
    for _ in 0..8 {
      let mask = (crc & 1).wrapping_neg();
      crc = (crc >> 1) ^ (0x82f6_3b78u32 & mask);
    }
  }
  !crc
}

#[cfg(test)]
mod tests {
  use super::*;

  #[test]
  fn header_round_trip() {
    let payload = b"HELLO-CSDP";
    let header = FrameHeader::new(
      MessageType::Data,
      0x1122_3344_5566_7788,
      7,
      1,
      0xAABB_CCDD_EEFF_0011,
      3,
      payload,
    );
    let wire = header.encode();
    let decoded = FrameHeader::decode(&wire).unwrap();
    decoded.verify_payload(payload).unwrap();
    assert_eq!(decoded.object_id, 0xAABB_CCDD_EEFF_0011);
    assert_eq!(decoded.chunk_id, 3);
    assert_eq!(decoded.payload_len, payload.len() as u64);
  }
}

```

The reference CRC routine is intentionally simple. A production implementation SHOULD use a hardware-accelerated CRC32C implementation where available.

19. CSDP Test Vector

For:

```

version = 1.0
message_type = DATA (0x0030)

```

```

session_id = 0x1122334455667788
stream_id = 7
sequence = 1
object_id = 0xAABBCCDDEEFF0011
chunk_id = 3
payload = ASCII "HELLO-CSDP"

```

The payload CRC32C is:

```
0x1162B70A
```

The complete 64-byte header is:

```

43 53 44 50 01 00 08 00
30 00 00 00
88 77 66 55 44 33 22 11
07 00 00 00
01 00 00 00 00 00 00 00
11 00 FF EE DD CC BB AA
03 00 00 00
00 00 00 00
0A 00 00 00 00 00 00 00
0A B7 62 11
24 DE CE 37

```

where 24 DE CE 37 is the little-endian representation of the header CRC32C 0x37CEDE24.

This vector can be used to validate independent implementations before transport integration.

20. Reference Processing Pipeline

20.1 Sender

```

MemoryX index
|
v
Extent scheduler
|
v
NUMA-local buffer
|
v
build CSDP header
|
v
RDMA/TCP/DWL transport
|
v
receiver

```

The sender MUST check credit before posting each DATA operation.

20.2 Receiver

```

receive frame
|
v
validate fixed header

```

```

    |
    v
validate epoch / stream
    |
    v
validate CRC32C
    |
    v
place raw bytes
    |
    v
update ack / credits
    |
    v
consumer / WSE staging

```

No generic object graph needs to exist for the payload.

21. MemoryX Streaming Scheduler Proposal

A MemoryX implementation can maintain a small state machine per object:

```

pub struct StreamState {
    pub object_id: u64,
    pub total_bytes: u64,
    pub next_chunk: u32,
    pub credits_bytes: u64,
    pub credits_chunks: u32,
    pub in_flight: u32,
    pub committed_chunks: u32,
}

```

The scheduler should combine:

- model execution order;
- chunk residency;
- receiver credit;
- storage queue depth;
- NUMA locality;
- network queue depth;
- collective fan-out.

The scheduler SHOULD prefetch the next tensor while the current tensor is being consumed.

22. Future Extensions

22.1 Sparse tensor encoding

CSDP can add a tensor descriptor extension for block-sparse data:

```

block_size
index_dtype
index_count
value_layout

```

The transport framing remains unchanged.

22.2 Quantized formats

The protocol should represent FP8/INT8/INT4 data as native tensor layouts rather than generic compressed blobs where possible.

22.3 GPU/accelerator memory registration

The same transport abstraction can be extended to memory registration handles for heterogeneous devices.

22.4 Programmable fabric operations

A future CSDP version can advertise fabric operations beyond fixed collectives:

```
replicate
reduce
filter
repartition
checksum
compress
```

The control plane would describe these operations while the bulk payload remains raw or structurally described.

22.5 Direct-Wafer specialization

A Direct-Wafer transport adapter could bypass Ethernet framing while preserving the same CSDP logical object model. This requires vendor/device-specific work and is not specified here because public documentation does not provide the complete Direct Wafer Link programming model.

23. Risks and Open Questions

23.1 Public-information gap

The exact internal architecture and wire protocol of MemoryX and SwarmX are not publicly documented at the level needed to claim direct compatibility. CSDP should therefore be treated as a clean-room proposal.

23.2 Serialization benchmark risk

A different metadata serializer may deliver little measurable benefit if tensor bytes dominate the workload. Benchmarks must focus on end-to-end data movement.

23.3 RDMA deployment complexity

RDMA introduces operational requirements around provisioning, congestion control, memory registration, NUMA affinity, and fault handling. A TCP profile remains valuable as a reference implementation and fallback.

23.4 Security/performance trade-off

AEAD on the highest-bandwidth path can increase CPU or accelerator work. The protocol therefore separates trusted-fabric and secure-data-plane profiles rather than forcing one security mode on every deployment.

23.5 Determinism versus collective performance

Floating-point reductions can be order-sensitive. Deterministic mode should therefore be explicit and measurable rather than assumed.

24. Implementation Roadmap

Phase 1 - Reference protocol

Implement:

- frame encoder/decoder;
- CRC32C;
- Postcard control messages;
- TCP transport;
- state machines;
- test vectors;
- fuzzing and malformed-frame rejection.

Phase 2 - MemoryX simulator

Implement:

- append-only extent store;
- rkyv/Serde metadata index;
- prefetch scheduler;
- double/triple buffer model;
- credit flow control.

Phase 3 - RDMA prototype

Implement:

- registered buffer management;
- RDMA send/receive or RDMA write profile;
- completion queues;
- NUMA affinity;
- queue-depth tuning.

Phase 4 - Collective engine

Implement:

- broadcast tree;
- reduce tree;
- all-reduce;
- topology descriptors;
- deterministic mode.

Phase 5 - Hardware integration

Map transport and buffer primitives to the target Cerebras platform and validate:

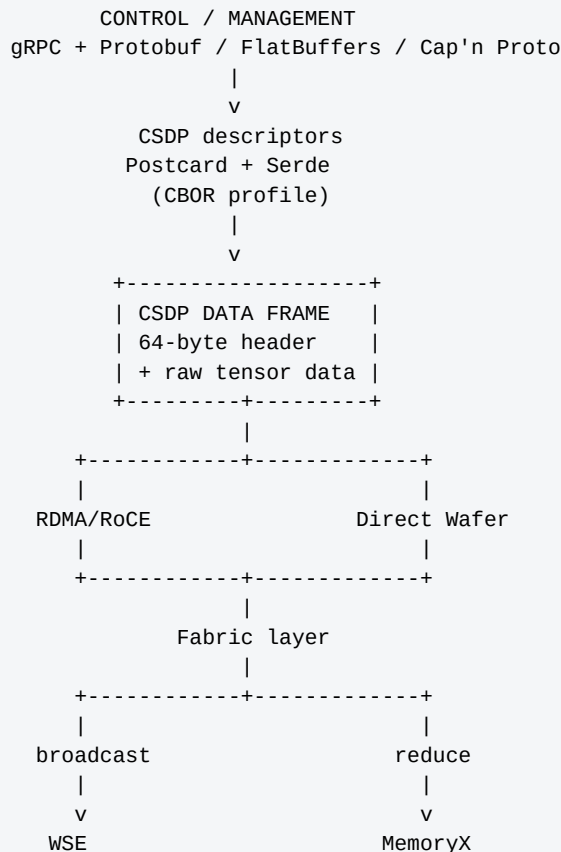
```
MemoryX -> CSDP -> fabric -> WSE
WSE -> CSDP -> reduce -> MemoryX
prefill -> CSDP -> KV cache -> decode
```

This phase is where platform-specific device and fabric programming information becomes essential.

25. Final Design Position

CSDP should not be positioned as another general-purpose serialization library. Its architectural purpose is to create an explicit boundary between **metadata serialization** and **tensor movement**.

The design can be summarized as:



The key architectural improvement is the removal of unnecessary serialization and copying on the hot path. The protocol still benefits from mature serialization formats where schemas, portability, and control messages matter.

For a Rust-first implementation, the strongest composition is:

Serde + Postcard for compact control, rkyv for local metadata, CSDP's fixed binary frame for tensors, CRC32C for frame integrity, BLAKE3/SHA-256 for end-to-end object integrity, and RDMA/RoCE as the primary external data transport with a TCP reference path.

This architecture is compatible with the public direction of Cerebras toward programmable I/O, RoCE-based connectivity, low-latency wafer interconnection, and heterogeneous/disaggregated AI systems while remaining explicit that CSDP is a new proposal rather than a disclosed Cerebras internal protocol.

Appendix A. Compact CSDP Wire Reference

CSDP frame		
0x00	magic[4]	"CSDP"
0x04	version_major[1]	
0x05	version_minor[1]	
0x06	header_len_words[1]	
0x07	flags[1]	
0x08	message_type[2]	
0x0A	status[2]	
0x0C	session_id[8]	
0x14	stream_id[4]	
0x18	sequence[8]	
0x20	object_id[8]	
0x28	chunk_id[4]	
0x2C	reserved[4]	
0x30	payload_len[8]	
0x38	payload_crc32c[4]	
0x3C	header_crc32c[4]	
0x40	optional extensions	
	payload	

All fixed-width fields are little-endian.

Appendix B. Recommended Default Parameters

Parameter	Draft default
Base header	64 bytes
Minimum chunk size tested	1 MiB
Candidate chunk sizes	1, 4, 8, 16, 64 MiB
Maximum protocol payload	64 MiB by default; implementation MAY increase
Frame integrity	CRC32C
Object digest	BLAKE3 or SHA-256
Control encoding	Postcard/Serde
Portable metadata encoding	CBOR
Local metadata archive	rkyv or equivalent
Primary transport	RDMA/RoCE v2
Compatibility transport	TCP
Flow control	byte + chunk credits
Default ordering	ordered stream
Duplicate behavior	idempotent discard
Epoch fencing	required
Deterministic reduce	opt-in

These defaults are starting points, not measured optimums.

Appendix C. Glossary

CSDP - Cerebras Distributed Streaming Protocol, the proposed protocol in this report.

MemoryX - Cerebras technology used as an off-wafer memory/storage and intelligent weight-streaming service in public Weight Streaming documentation.

SwarmX - Cerebras scale-out broadcast/reduce fabric described in public documentation for Weight Streaming clusters.

WSE - Wafer-Scale Engine.

WSE-3 / WSE-3T - third-generation WSE and its Turbo variant used in CS-3 and CS-4 respectively.

Nexus - Cerebras rack-scale platform architecture introduced with CS-4.

Direct Wafer Links - Cerebras-described switch-free wafer interconnect capability in CS-4.

RoCE v2 - RDMA over Converged Ethernet v2.

Tensor descriptor - structured metadata describing tensor identity, shape, layout, dtype, size, and transfer properties.

Extent - a contiguous storage range used to hold a portion of a tensor/checkpoint object.

Epoch - cluster generation/fencing value used to prevent stale nodes from participating in a new execution generation.

Appendix D. References

[R1] Cerebras, “Linear Scaling Made Possible with Weight Streaming.” Public description of MemoryX + SwarmX and weight/gradient streaming. <https://www.cerebras.ai/blog/linear-scaling-made-possible-with-weight-streaming>

[R2] Cerebras Developer Documentation, “Weight Streaming Execution.” <https://training-api.cerebras.ai/en/1.9.1/wsc/cerebras-basics/cerebras-execution-modes.html>

[R3] Cerebras, “Scaling Up and Out: Training Massive Models on Cerebras Systems using Weight Streaming.” <https://www.cerebras.ai/blog/scaling-up-and-out-training-massive-models-on-cerebras-systems-using-weight-streaming>

[R4] Cerebras, “Cerebras Systems Unveils World’s Fastest AI Chip with Whopping 4 Trillion Transistors” (WSE-3). <https://www.cerebras.ai/press-release/cerebras-announces-third-generation-wafer-scale-engine>

[R5] Cerebras, “Cerebras CS-3: the world’s fastest and most scalable AI accelerator.” <https://www.cerebras.ai/blog/cerebras-cs3>

[R6] Cerebras, “Cerebras Unveils CS-4: Up to 30 Times Faster than GPU-based Solutions,” 2026-08-18. <https://investors.cerebras.ai/news-releases/news-release-details/cerebras-unveils-cs-4-30-times-faster-gpu-based-solutions>

[R7] Cerebras, “CS-4.” <https://www.cerebras.ai/cs4>

[R8] Cerebras, “Cerebras is coming to AWS,” 2026-03-13. <https://www.cerebras.ai/blog/cerebras-is-coming-to-aws>

[R9] Cerebras, “AMD and Cerebras Announce Industry-Leading Ultra-Low-Latency and High Throughput AI Inference Solution,” 2026-07-23. <https://investors.cerebras.ai/news-releases/news-release-details/amd-and-cerebras-announce-industry-leading-ultra-low-latency-and>

- [R10] Protocol Buffers, “Encoding.” <https://protobuf.dev/programming-guides/encoding/>
- [R11] gRPC, “Performance Best Practices.” <https://grpc.io/docs/guides/performance/>
- [R12] gRPC, “Flow Control.” <https://grpc.io/docs/guides/flow-control/>
- [R13] MessagePack, “MessagePack specification.”
<https://github.com/msgpack/msgpack/blob/master/spec.md>
- [R14] BSON, “Specification.” <https://bsonspec.org/spec.html>
- [R15] C. Bormann and P. Hoffman, RFC 8949, “Concise Binary Object Representation (CBOR).”
<https://www.rfc-editor.org/rfc/rfc8949.html>
- [R16] Cap'n Proto, “Encoding Specification.” <https://capnproto.org/encoding.html>
- [R17] FlatBuffers, “Overview” and “Rust.” <https://flatbuffers.dev/> and
<https://flatbuffers.dev/languages/rust/>
- [R18] Serde, “Serde.” <https://docs.rs/serde/latest/serde/>
- [R19] Postcard, “postcard - Rust.” <https://docs.rs/postcard/latest/postcard/>
- [R20] rkyv, “Archive.” <https://rkyv.org/architecture/archive.html>
- [R21] rkyv, “Format.” <https://rkyv.org/format.html>

Document Change Log

Version	Date	Change
1.0-draft	2026-10-03	Initial complete research report and formal Rust CSDP specification

Engineering Status and Recommended Next Step

Recommended implementation sequence: reference CSDP/TCP -> MemoryX simulator -> benchmark serializers vs raw framing -> RDMA/RoCE transport -> collective engine -> target hardware integration. The first measurable milestone should be an end-to-end tensor transfer benchmark that reports CPU cycles/byte, memcpy volume, RDMA throughput, and compute stall percentage rather than serializer throughput alone.